

Overview

Part 16 —  AI Tech Stack · 16-ai-tech-stack/README.md

Module 16 — AI Tech Stack

Where you are: Stop 16 of 16 — the capstone module. **Time:** ~60–70 hours across 7 tracks. **Prereq:** Modules 01–09 give you the theory; this module is where you actually build things.

Every other module in this track teaches a concept: how a transformer works, what RAG is, how agents plan. This module teaches the **languages, frameworks and runtimes** you use to ship that theory as software — end to end, from `import numpy` to a deployed agent on managed cloud infrastructure.

Each track below is a **self-contained learning unit**: a README, two notes files, runnable code (verified offline wherever the underlying product allows it), a lab, papers/primary sources, and a slide deck. Work them in order if you're new to the space, or jump to whichever one blocks you today.

The seven tracks, in the order most people actually need them

#	Track	What it gives you	Time
16.1	 Python [16-ai-tech-stack/tracks/python]	Modern tooling (uv, ruff), async for LLM batching, Pydantic for structured output	8–10h
16.2	 TypeScript [16-ai-tech-stack/tracks/typescript]	The application layer: streaming, typed tool calls, Zod, the Vercel/Anthropic SDKs	8–10h
16.3	 LangChain [16-ai-tech-stack/tracks/langchain]	The Runnable interface, LCEL, tools, retrieval — and when to skip the framework	8–10h
16.4	 LangGraph [16-ai-tech-stack/tracks/langgraph]	Agents as stateful graphs: persistence, human-in-the-loop, multi-agent patterns	8–10h
16.5	 LangSmith [16-ai-tech-stack/tracks/langsmith]	Tracing, LLM-as-judge evals, the trace→dataset→fix→ship operating loop	6–8h
16.6	 AWS Bedrock [16-ai-tech-stack/tracks/aws-bedrock]	Shipping AI on AWS: Converse API, Knowledge Bases, Guardrails, IAM, cost	8–10h
16.7	 Claude Code & Agent SDK [16-ai-tech-stack/tracks/claude-code]	Using the harness well, then building your own agent on the SDK	6–8h

Why this order: Python and TypeScript are the substrate everything else sits on. LangChain gives you the model/tool abstraction; LangGraph is the runtime you graduate to once a chain needs a loop or an approval gate; LangSmith is how you see whether any of it is actually working. Bedrock is one concrete answer to

"where does this run in production"; Claude Code and its SDK are both a tool you use daily and a second, provider-native way to build the same kind of system.

You do not need all seven to ship something real. A working RAG service needs Python (or TypeScript) + LangChain-or-not + somewhere to run it + a way to see what it's doing. That's four tracks, not seven.

How each track is organized

```
tracks/<name>/
├── README.md           front door: objectives, path, exit check
├── notes/              core concepts + practitioner/production craft
├── code/               runnable examples – offline wherever possible
├── lab/EXERCISES.md    graded exercises with explicit deliverables
├── papers/PAPERS.md    official docs and primary sources, verified URLs
├── resources/RESOURCES.md courses, repos, communities
└── slides/             deck.json source + built .pptx + Marp slides.md
```

Cross-links back to the theory

This module is deliberately hands-on; the *why* lives elsewhere in the track:

- Agent theory and the ReAct loop → ../06-ai-agents/
- Multi-agent topology and when it earns its cost → ../07-agentic-ai/
- Retrieval quality, chunking, reranking → ../08-rag/
- The Model Context Protocol itself → ../09-mcp/
- Reference architectures for the systems these tools build → ../10-ai-architecture/
- Prompt injection, the lethal trifecta, red-teaming → ../13-ai-security/
- Human-oversight and audit requirements for approval gates → ../12-ai-governance/
- Building the eval sets these tools consume → ../15-ai-evals/

Exit check

Pick one real (or realistic) use case and build a small end-to-end slice: a typed client call, a retrieval step, a tool-using loop with at least one cycle, traced, with one automated eval, running somewhere other than your laptop. If you can point at each of those five pieces in your own code and say which track taught you the pattern, you're done with this module.